SD242 – Verificação de Sistemas

Autores	
Anderson Max Almeida e Porfirio (Hbr)	Nome 2 (INSTITUIÇÃO)
Nome 3 (INSTITUIÇÃO)	Nome 4 (INSTITUIÇÃO)

Histórico de revisões		
05/11/2025	V1.0	Versão inicial

Página em branco.

Aula 32
Asserções e verificação formal

Tópicos

- Introdução
- Assertion Statement
- Immediate vs Concurrent
- Formal Verification

Introdução

- Uma assertion é uma forma de especificar um comportamento esperado do sistema.
- É usada para verificar automaticamente se o comportamento do design está de acordo com uma propriedade especificada.
- Quando a propriedade é falsa, ocorre uma falha da assertion.

- Uma declaração de asserção procedural:
  - Semelhante a uma declaração if.
  - Ignorada pela síntese.
  - É executada quando o fluxo procedural chega à instrução assert.

Introdução

```
always @(negedge clk)
    A1: assert ( !(wr_en && rd_en) )
```

```
always @(negedge clock)
  if (wr_en && rd_en) $display("error");
```

wr_en	rd_en	wr_en && rd_en	!(wr_en && rd_en)	Assertion
0	0	0	**1**	Passa
0	1	0	**1**	Passa
1	0	0	**1**	Passa
1	1	1	**0**	Falha → $error

```
[label:] assert (expressão_booleana)
  [pass_statement]
  [else fail_statement];
```

```
A1: assert ( ~(wr_en && rd_en) ) else $error("Conflito rd/wr: wr_en=%b, rd_en=%b", wr_en, rd_en);
```

Assertion Statement

- Uma assertion aparece como um assertion statement
- Define a função de verificação a ser executada

- Tipos de statements:
  - **Assert** - Obriga o design a satisfazer a property
  - **Assume** - Define suposições sobre o environment
    Ferramentas formais usam a property para gerar estímulos

  - **Cover** - Coleta coverage da property
  - **Restrict** - Impõe constraints para verificação formal
    Define uma property como **constraint** para formal verification

- Quando uma assertion falha, o simulador deve exibir as seguintes informações mínimas:
  - O nível de severidade da asserção.
  - O nome do arquivo e o número da linha da asserção.
  - O tempo de simulação em que a asserção falhou.
  - O rótulo da asserção ou, se não rotulada, o escopo da asserção.

Immediate vs Concurrent

- Existem dois tipos de assertions:
- Immediate
  - Executadas como statements em blocos procedurais
  - Seguem a semântica de eventos da simulação
  - Destinadas primariamente à simulação
  - Sempre estão dentro de um fluxo procedural (Always, initial, ou evento)
  - Elas não possuem o conceito de clocks ou reset

- Concurrent
  - Baseadas em clock semantics
  - Usam valores amostrados (sampled values)
  - Avaliadas em ciclos de clock
  - Abstraem eventos entre bordas de clock
  - Muito usadas em formal verification

Immediate vs Concurrent

- Immediate

```
function automatic logic [7:0] soma( input logic [3:0] a, input logic [3:0] b );
        logic [4:0] result;
        result = {1'b0,a} + {1'b0,b};

    // Immediate Assertion: executada imediatamente como um statement procedural
        assert (!result[4]) else $error("Overflow");
        return result[3:0];
    Endfunction
```

- É executada no instante em que o código passa por aquela linha.
- Sempre estão dentro de um fluxo procedural.
- Incapazes de verificar comportamentos que se estendem ao longo do tempo.
- São vulneráveis aos **glitches** de "delta-time", onde podem ser afetadas por valores temporários no meio de um time-step.

Immediate vs Concurrent

- Immediate assertions são perfeitamente apropriadas quando queremos verificar uma condição no ponto em que o código procedural é executado.
    - Em funções ou outras estruturas verdadeiramente não sincronizadas, pode adicionar uma assertion imediata dentro de uma função, para verificar a segurança de seus argumentos ou conferir um cálculo.
    - dentro de always, initial, task ou function;
    - para verificar argumentos de uma função;
    - para verificar resultados de cálculos;
    - para verificar condições que devem ser verdadeiras no momento da execução do código.
- **Immediate assertions** devem ser usadas com **cautela** e são recomendadas apenas nos casos mencionados acima.

Immediate vs Concurrent

- Concurrent

```
// Concurrent Assertion: baseada em clock semantics
// Verifica se, após um sinal de 'request', o 'acknowledge' chega em exatamente 1 ciclo

property p_req_ack;
    @(posedge clk)
    req |-> ##1 ack;          // req |=> ack
endproperty

a2: assert property (p_req_ack)
    else $display("Falha: ACK não recebido após REQ no clock %0t", $time);
```

Immediate vs Concurrent

- Concurrent
  O system verilog assertion tem uma limguagem temporal para as concorrent assertions.
  Ex: `|-> |=> ##1 ##[1:3],` etc.

- Concurrent assertions permitem especificar e verificar **relações temporais entre sinais**.
- A maioria das **propriedades temporais utilizadas em FPV** é expressa por meio de concurrent assertions.
- Podem também incorporar condições de reset, por exemplo:

```
property p_req_ack;
    @(posedge clk)
    disable iff (!reset_n)
    req |-> ##1 ack;
endproperty
```

- Essas assertivas suportam a especificação clara de **clocks** e **resets** e são capazes de examinar o comportamento de valores ao longo do tempo.

Immediate vs Concurrent

	Immediate	**Concurrent**
Gatilho	Mudança em a ou b	Borda de subida do clk
Tempo	Ocorre no tempo zero do evento (Avaliação instantânea)	Avaliada ao longo de múltiplos ciclos
Valores	Usa o valor atual	Usa os **sampled values** de req e ack
Complexidade	Apenas condições booleanas simples	Permite sequências temporais ##1

Rótulos (Labels)

- [label:] assert (expressão_booleana) [pass_statement] [else fail_statement];

- Assertions podem executar código:
  - Em caso de sucesso (pass)
  - Em caso de falha (fail / else)

- Chamados de action blocks
  - Muito comum omitir o bloco de sucesso, mas pode ser utilizado para debug.
  - Apenas código de verificação, (fail) contar a falha ou exibir valores para debug.

Níveis de Severidade

- `$info`
- `$warning`
- `$error` (padrão)
  - Severidade pode ser sobrescrita no bloco de ação
  - Sintaxe igual ao `$display`
  - $error ("Assertion REQ_RSP passed at %t", $time);

- `$fatal` → termina a simulação
  - Severidade define **o impacto da falha**
  - Nem toda falha precisa parar a simulação

Operadores Temporais Comuns

Operador	Descrição	Exemplo
##n	Delay de n ciclos	req ##1 gnt
[*n]	Repetição exata	sig[*3]
[*m:n]	Repetição de m a n	sig[*1:5]
[+]	Uma ou mais vezes	sig[+]
[=n]	n vezes, não consecutivas	sig[=2]
|->	Overlapped implication	a |-> b
|=>	Non-overlapped implication	a |=> b
and / or	Operadores lógicos	a and b
throughout	Durante toda a sequência	en throughout seq

Assertion Guards

- Assertions podem ser desabilitadas usando iff(condition) construct.
- Lower Performance Version

```
property req_rsp;
  @(posedge clk);
  $(posedge req) |=>
  (req & ~rsp)[*1:2]
   ##1 (req && rsp)[->1]
   ##1 (~req && ~rsp);
endproperty: req_rsp
```

- Higher Performance Version

```
property req_rsp;
  // Disable if reset is active:
  @(posedge clk) iff(!reset);
  $rose(req) |=>
  (req & ~rsp)[*1:4]
   ##1 (req && rsp)
   ##1 (~req && ~rsp);
endproperty: req_rsp
```

Evite usar mensagens Pass e Fail (Uvm-CookBook)

- Lower Performance Version

```
REQ_RSP: assert property(req_rsp) begin
        $display("Assertion REQ_RSP passed at %t", $time);
         else
         $display("Error: Assertion REQ_RSP failed at %t", $time);
End
```

- Higher Performance Version
  - REQ_RSP: assert property(req_rsp) ;

- Diretivas de Compilação

```
`ifndef SYNTHESIS
   // Código de assertions aqui
   // (ignorado durante síntese)
`endif
```

Interface UVM (uvm_interface)

```
interface bus_if (input clk, input rst_n);
    logic [31:0] addr;
    logic [31:0] data;
    logic wr_en;
    logic rd_en;

    // ASSERTIONS na interface
    property p_no_read_write_conflict;
        @(posedge clk) disable iff (!rst_n)
        !(wr_en && rd_en);
    endproperty

    assert_no_conflict: assert property (p_no_read_write_conflict)
        else `uvm_error("BUS_IF", "Conflito read/write simultâneo")

    // Coverage também na interface
    covergroup bus_cg @(posedge clk);
        // ...
    endgroup
endinterface
```

Driver/Monitor/Scoreboard

```
class bus_driver extends uvm_driver #(bus_transaction);
    virtual bus_if vif;

    task run_phase(uvm_phase phase);
        forever begin
            seq_item_port.get_next_item(req);

            // Assertion procedural no driver
            driver_check: assert (req.addr < 32'h1000)
                else `uvm_error("DRIVER", "Endereço fora do range")

            drive_transaction(req);
            seq_item_port.item_done();
        end
    endtask
endclass
```

Verificar o RTL diretamente

```
4  // assertions.sv (arquivo separado)
5  module bus_assertions(
6      input logic clk,
7      input logic rst_n,
8      input logic [31:0] addr,
9      input logic wr_en,
10     input logic rd_en
11 );
12
13     // Concurrent assertions
14     property p_valid_addr;
15         @(posedge clk) disable iff (!rst_n)
16         (wr_en || rd_en) |-> (addr < 32'h1000);
17     endproperty
18
19     assert_valid_addr: assert property (p_valid_addr)
20         else $error("Endereço inválido: %h", addr);
21
22     property p_no_deadlock;
23         @(posedge clk) disable iff (!rst_n)
24         wr_en |=> ##[1:10] !wr_en;
25     endproperty
26
27     assert_no_deadlock: assert property (p_no_deadlock)
28         else $error("Possível deadlock detectado");
29
30 endmodule
```

Conclusão

- Assertions em UVM seguem o princípio de separação de responsabilidades - cada tipo de verificação vai no local mais apropriado, permitindo reuso, manutenção e debug eficientes.

  - Protocolo → Interface
  - Estrutura → Bind files
  - Comportamento → Classes UVM
  - Sempre use labels descritivos
  - Integre com reporting UVM (uvm_error, uvm_warning)

Laboratório

- Assertions - FIFO

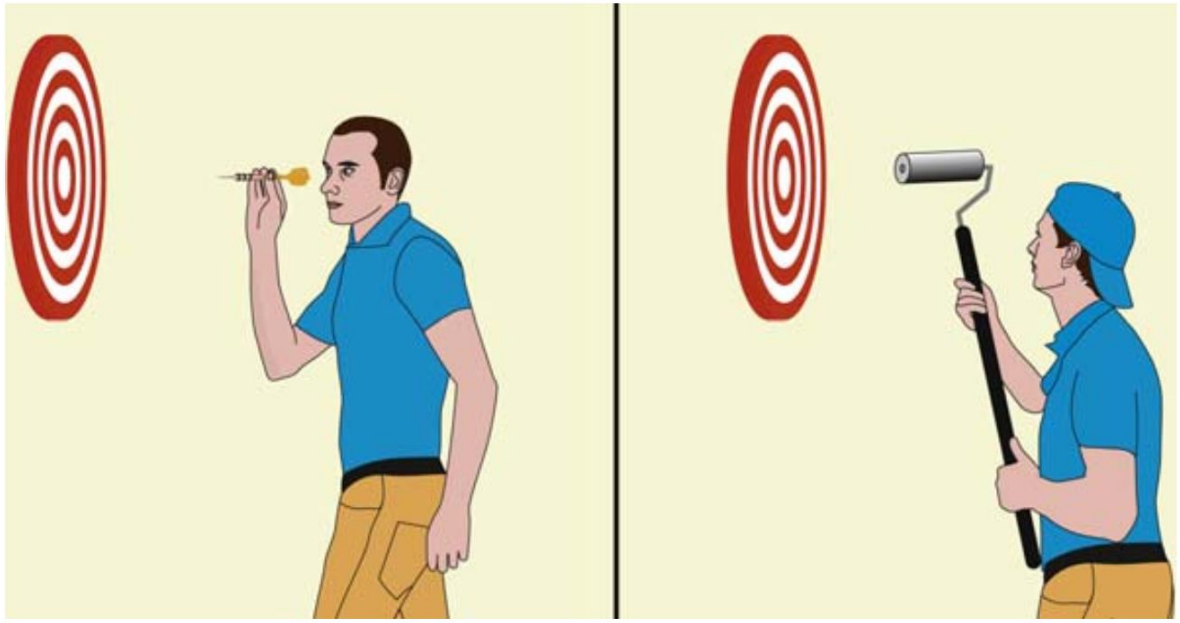
Introdução

- Formal Verification (Verificação Formal)
  - Uso de ferramentas que analisam matematicamente o espaço de possíveis comportamentos de um design, dentro das hipóteses/assumptions e do modelo.

- Simulação
  - Explora o design por meio de um conjunto finito de estímulos e cenários.
  - É como lançar uma quantidade finita de dardos sobre o espaço de comportamentos possíveis.
    - direcionados; aleatórios; constrained-random;
- Formal Verification
  - Formal busca provar matematicamente uma propriedade sobre todo o espaço de comportamentos permitido pelo modelo e pelas restrições.

Simulação x Formal Verification

- Cobertura do seu design por simulação e verificação formal.
- A simulação é como jogar dardos em um alvo, enquanto o FV cobre todo o seu alvo com um rolo de tinta.



Problemas que a FV Resolve

- Erros caros em Corner Cases (casos de canto) não atingidos pela simulação.
- Permitir a verificação de propriedades de um módulo mesmo antes de o ambiente completo de simulação estar disponível.
- Garantir que otimizações de RTL, síntese ou FEV/equivalence checking não alteraram a funcionalidade pretendida.
- Quando simulações aleatórias param de encontrar bugs novos no final do projeto. (coverage closure)

O Caso Pentium FDIV

- Em 1994, usuários descobriram que a FPU do Intel Pentium tinha um defeito de projeto.
- Em certas divisões de ponto flutuante, o resultado podia apresentar um erro pequeno, mas significativo, geralmente a partir da quarta casa decimal.
- Um defeito na tabela de lookup do algoritmo de divisão SRT (usado para alta performance). Foram omitidos 5 dos 1066 valores na tabela, causando arredondamento incorreto em casos específicos.
- (4.195.835 / 3.145.727) * 3.145.727 - 4.195.835 = 0, mas o Pentium ˜= 256
- A Intel ofereceu trocar o processador de qualquer usuário que solicitasse, sem necessidade de comprovação de uso crítico.
- Prejuízo de USD 475 milhões (aprox. USD 1 bilhão hoje).
- **Conclusão:** Em área crítica, a FV é essencial para prevenir recalls bilionários. A simulação falhou em "jogar o dardo" no caso exato da divisão.

Limitações Matemáticas Fundamentais

- A FV lida com fronteiras da ciência da computação:
  - A Formal Verification não é uma solução universal para qualquer problema.
  - Existem limitações teóricas e computacionais para a verificação automática de sistemas arbitrários.
  - O Halting Problem, de Turing, demonstra que não existe um algoritmo geral capaz de determinar, para qualquer programa e qualquer entrada, se sua execução terminará.
  - Os Teoremas da Incompletude de Gödel mostram limitações fundamentais de sistemas formais suficientemente expressivos e consistentes.

Limitações Matemáticas Fundamentais

- Na prática, a verificação formal também enfrenta problemas como:
  - explosão do espaço de estados;
  - complexidade computacional;
  - propriedades difíceis de provar;
  - necessidade de abstrações e restrições;
  - recursos computacionais finitos.

Limitações Matemáticas Fundamentais

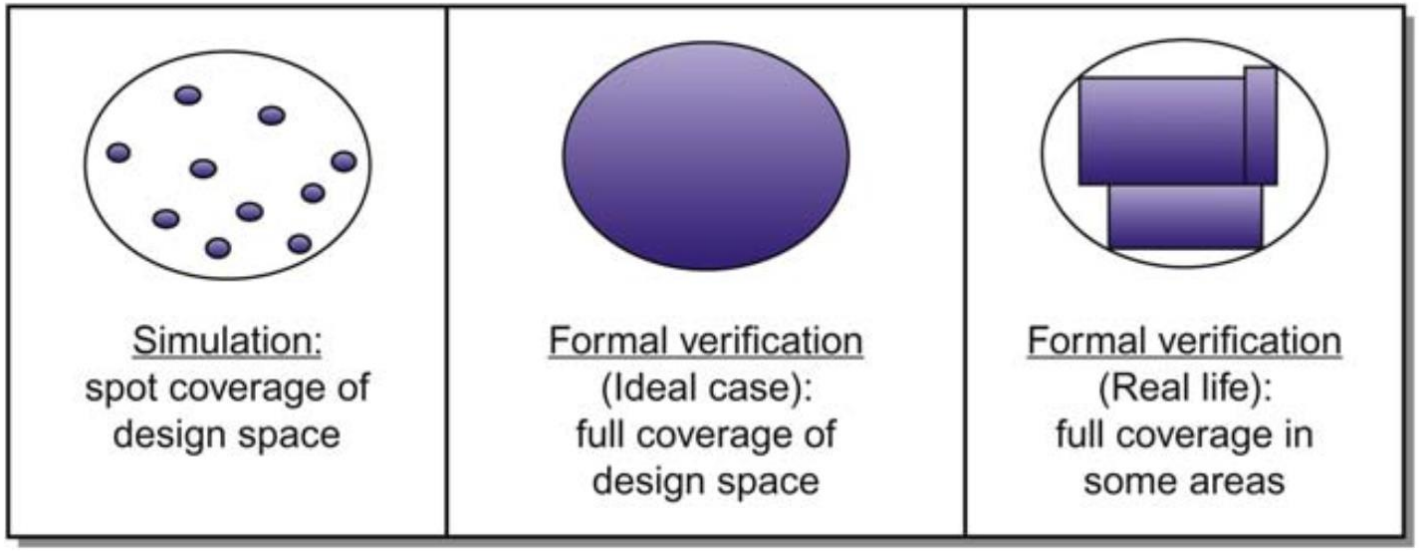
- Por causa dessas barreiras teóricas, não existe uma ferramenta de FV "universal e perfeita" que prove qualquer propriedade em qualquer sistema.
- Contudo, para designs finitos e específicos (como um chip, um protocolo), a FV é extremamente eficaz e essencial — pois lida com sistemas delimitados onde essas limitações podem ser contornadas na prática.

Complexidade e o Problema SAT

- SAT (Satisfiability Problem): Dada uma expressão booleana, existe uma combinação de variáveis que a torne verdadeira?
  - Ex.: ((A OU B) E (NÃO A)), existe uma combinação Verdadeiro/Falso para as variáveis que torne toda a fórmula VERDADEIRA?
- NP-Complete: O SAT é provadamente difícil. O tempo de solução cresce exponencialmente (2^n).
  - Não se conhece (e acredita-se ser impossível) um algoritmo eficiente para resolver todos os casos do SAT no pior cenário.
- O Exemplo do Somador: Testar todas as entradas de um somador de 32 bits levaria mais de 500 mil anos via simulação.

Heuristics (Heurísticas) e a "Mágica" da FV

- Ferramentas modernas usam heurísticas (estratégias de resolução) que funcionam na vasta maioria dos designs RTL industriais.
- ROI (Retorno sobre Investimento)
  - Mesmo sem prova 100%, a FV cobre um subconjunto do espaço exponencialmente maior que a simulação.



Abstrações e Simplificações (Mecânicas)

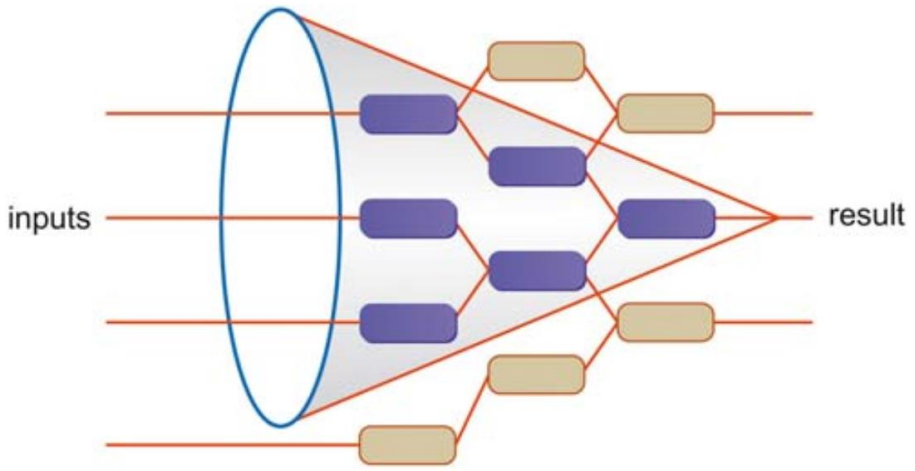
- Como tornar problemas complexos em problemas tratáveis:
- Proof Decomposition (Decomposição de Prova)
  - Dividir propriedades complexas em partes menores.
- Targeted Verification (Verificação Direcionada)
  - Focar a ferramenta em áreas de risco (ex: fila quase cheia).
- Size Reductions (Reduções de Tamanho)
  - Reduzir largura de barramentos de dados para focar na lógica de controle.
- Case Splitting (Divisão de Casos)
  - Verificar cada modo ou opcode separadamente.

Técnicas de Abstração Avançadas

- Design Abstraction
  - Ignorar detalhes (ex: tratar um contador como gerador de valores aleatórios).
- Semi-Formal Verification
  - Combina simulação para atingir estados profundos com FV para cobrir a vizinhança.
- Bounded Proofs (Provas Limitadas)
  - Provar a correção para um número fixo de ciclos (ex: 100 ciclos).

Cone of Influence (Cone de Influência)

- Remoção automática da lógica que não afeta matematicamente o resultado da propriedade verificada.
- Reduz drasticamente a quantidade de lógica que os algoritmos precisam processar.
  - Exemplo de redução por Cone de Influência



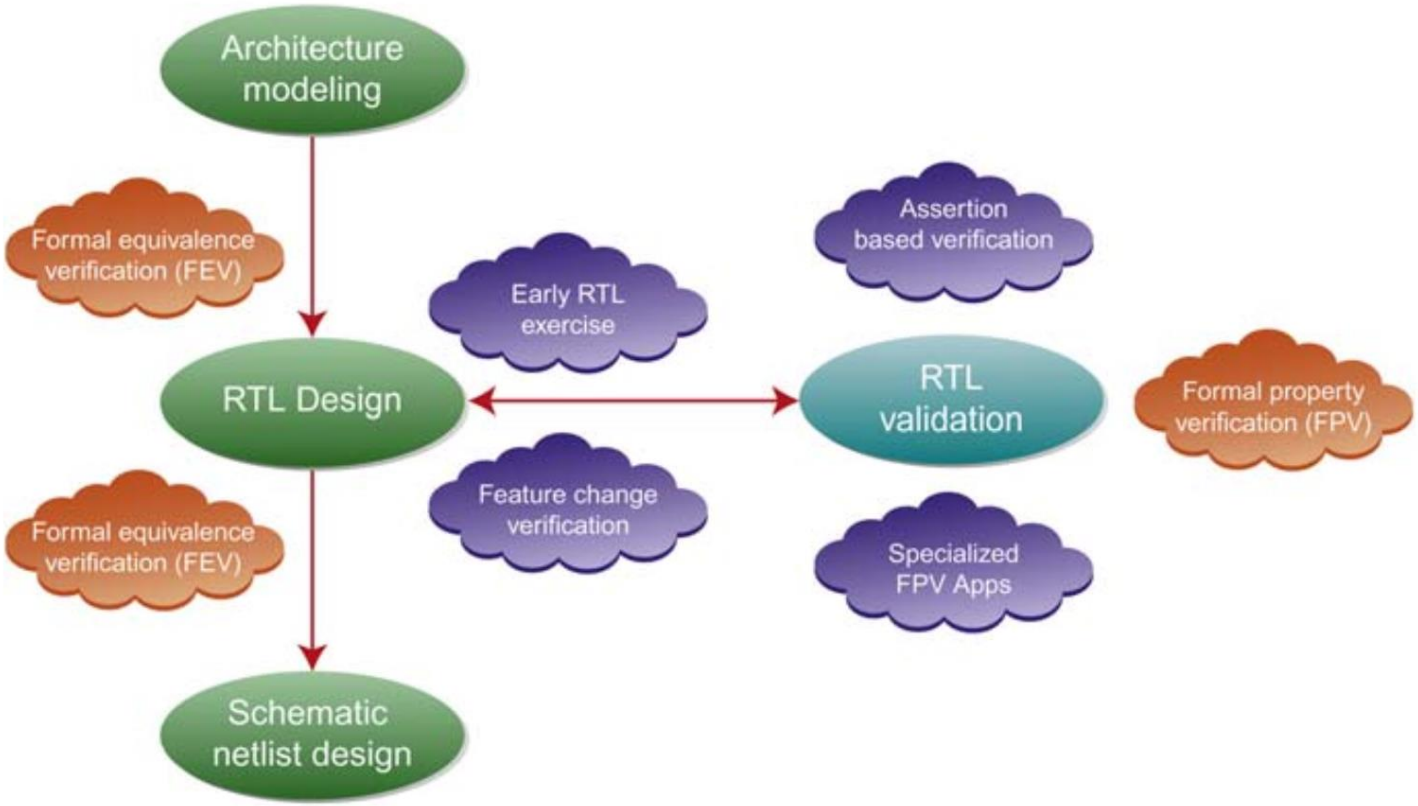
Cone of influence

FPV - Formal Property Verification

- Early Design Exercise
  - Achar bugs no RTL antes de ter um ambiente de simulação pronto.
- Signoff Quality FPV
  - Substituir a simulação em blocos críticos para garantir 100% de implementação da especificação.
- Unreachable Coverage
  - Identificar partes do código que a simulação não atinge por serem logicamente impossíveis.

FPV - Formal Property Verification

- Early Design Exercise
- Signoff Quality FPV
- Unreachable Coverage



Formal Equivalence Verification

- RTL-to-Netlist
  - Comparar o RTL com o circuito de portas lógicas (Netlist) gerado pela síntese.
- Feature Change Verification
  - Garantir que otimizações (timing/área) não quebraram a funcionalidade original.
- State Matching
  - Usar a correspondência dos *flops* entre os modelos para simplificar a prova matemática.

Tabelas Verdade e Crescimento Exponencial

- A lógica booleana pode ser representada por tabelas verdade.
- Cada nova entrada dobra o número de linhas.

Table 2.1 Truth table for simple ADD.

Stimulus		Result	
X	**y**	**Sum**	**Carry**
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 2.2 Truth table for simple MUX.

Stimulus			Result
x	**y**	**z**	**MUX**
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

O Exemplo da Vending Machine

- Provar que a máquina aceita 35 centavos (moedas de 5, 10 e 25).
- Sparsity (Esparsidade): De 64 combinações possíveis, apenas 6 são válidas. A FV foca nessas 6 em vez de olhar todas as 64.
- Estruturas como BDDs(binary decision diagrams) e SAT exploram padrões e simetrias para análise compacta.

Table 2.3 Efficient truth table for 35-cent vending machine.

Stimulus			Result
Nickels [2:0]	**Dimes [1:0]**	**Quarters**	**Vend**
3'b000	2'b01	1	1
3'b010	2'b00	1	1
3'b001	2'b11	0	1
3'b011	2'b10	0	1
3'b101	2'b01	0	1
3'b111	2'b00	0	1
Any other pattern			0

Lógica Sequencial e Álgebra Booleana

- Sequential Logic: Uso de marcas de linha ($') para representar o próximo estado em tabelas verdade.
- DNF (Disjunctive Normal Form): Representação em "soma de produtos" usada pelas ferramentas para automatizar a construção de fórmulas.

Table 2.4 Truth table with sequential logic.

Stimulus		Result	
Input	**Sum**	**Sum'**	**Carry'**
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 2.5 Truth table for circuit example.

Stimulus			Intermediates			Result
x	***y***	***Z***	***a = NOT y***	***b = AND(x,y)***	***c = AND(a,z)***	***OR(a,c)***
0	0	0	1	0	0	0
0	0	1	1	0	1	1
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	1
1	1	1	0	1	0	1

Lógica Sequencial e Álgebra Booleana

- Booleans
  - São expressões booleanas padrão do SystemVerilog.
  - Uma expressão booleana pode ser uma única variável lógica ou uma fórmula
    - !(gnt[0] && !req[0]))
    - (req[0]&&req[1]&&req[2]&&req[3])
- Sequences
  - São declarações sobre valores booleanos (ou outras sequências) acontecendo ao longo do tempo.
  - Elas dependem de um evento de clock (clocking event) claramente definido para determinar a passagem do tempo.
    - Aseguinte sequência representa uma requisição seguida por uma concessão (grant) em dois ciclos de clock
    - req[0] ##2 gnt[0]

Lógica Sequencial e Álgebra Booleana

- Properties
  - Combinam sequências com operadores adicionais para mostrar implicação e conceitos semelhantes, a fim de expressar algum comportamento que se espera manter em um projeto.
    - Uma propriedade pode estabelecer que, se recebermos uma requisição seguida dois ciclos depois por uma concessão, isso implica que a concessão será desativada no ciclo seguinte
    - req[0] ##2 gnt[0] |-> ##1 !gnt[0]
- Assertion statements
  - São declarações que utilizam uma das palavras-chave assert, assume ou cover, fazendo com que uma propriedade SVA seja verificada como uma assertiva, suposição ou propriedade de cobertura.
  - Uma propriedade não tem efeito a menos que seja usada em algum tipo de declaração de assertiva.
    - gnt_falls: assert property(req[0] ##2 gnt[0] |-> ##1 !gnt[0]);

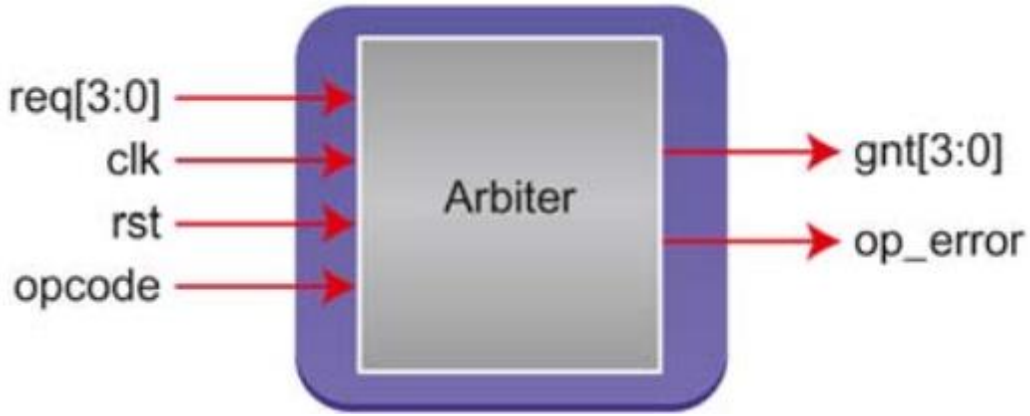
SystemVerilog Assertions

- Assertion é um statement (declaração) sobre o design que se espera que sempre seja Verdade.
- O Árbitro (Exemplo)
- Nunca se espera um gnt para agente 0 quando este não é pedido.

  check_grant: assert property (!(gnt[0] && !req[0]))

  else $error(“Grant without request for agent 0!”);

  - Simulação com assertions.
  - Rodar uma FPV, assertions são os alvos.



Assert, Assume e Cover

- Assertion (Assertiva): Declaração que deve ser sempre verdadeira sobre o design sob teste.

- Assumption (Suposição): Restringe entradas do ambiente (axioma). O que a ferramenta assume ser verdade sobre as entradas.

```
good_opcode: assume property (opcode != RESERVED)
    else $error("Illegal opcode.");
```

- Cover (Cobertura): Identifica estados que podem ocorrer. Previne o Overconstraint (restrição excessiva).

```
cover_all_at_once: cover property
    (req[0]&&req[1]&&req[2]&&req[3]);
```

Assim,

- A FV transforma a incerteza da simulação em near-certainty (quase certeza) matemática.
- O sucesso depende do engenheiro saber usar as Heurísticas e as Abstrações corretas.
- FV é um kit de ferramentas geral para interagir, verificar e entender designs RTL modernos.

Assim,

- A FV transforma a incerteza da simulação em near-certainty (quase certeza) matemática.
- O sucesso depende do engenheiro saber usar as Heurísticas e as Abstrações corretas.
- FV é um kit de ferramentas geral para interagir, verificar e entender designs RTL modernos.

Referência

- https://www.perlego.com/book/1874638/formal-verification-an-essential-toolkit-for-modern-vlsi-design-pdf
- Formal Verification - An Essential Toolkit for Modern VLSI Design

Second Edition - Erik Seligman, Tom Schubert M V Achutha Kiran Kumar